NATIONAL INSTITUTE OF TECHNOLOGY SRINAGAR

Department of Electronics and Communication Engineering

Design and Implementation of a 32-bit Arithmetic Logic Unit using Verilog HDL

Mini Project Report

Submitted by

Mohammad Irtiqa Bhat
(2024-25MECECS005)

Mohammad Abbas
(2025MECECS002)

Department of Electronics and Communication Engineering
National Institute of Technology Srinagar

2026

Abstract

This report presents the design, implementation and verification of a 32-bit Arithmetic Logic Unit (ALU) in Verilog HDL. The ALU performs sixteen arithmetic, logic, shift and compare operations and produces four status flags: Zero, Negative, Carry and Overflow. The adder is built structurally from eight 4-bit carry-lookahead blocks, shifting is done by a five-stage barrel shifter, and the compare operations re-use the subtractor so that no separate comparator is needed. The design is verified with self-checking testbenches that combine an independent behavioural reference model, directed corner-case tests, constrained-random tests and golden vectors generated by a Python model. Yosys scripts are provided for generic gate-level and Lattice ECP5 synthesis.

Contents

1 Introduction **2**
1.1 Objectives . . . 2
1.2 Tools . . . 2

2 Theory **2**
2.1 Two's complement arithmetic . . . 2
2.2 Carry-lookahead addition . . . 2
2.3 Status flags . . . 3
2.4 Comparison using the subtractor . . . 3
2.5 Barrel shifter . . . 3

3 Design **3**
3.1 Interface . . . 3
3.2 Operation set . . . 4
3.3 Architecture . . . 4
3.4 Adder . . . 6
3.5 Barrel shifter . . . 6

4 Verilog implementation **6**

5 Verification **7**
5.1 Method . . . 7
5.2 Test plan . . . 7
5.3 Golden-model cross-check . . . 7
5.4 Known-answer tests . . . 8

6 Simulation and synthesis **8**
6.1 Running the simulation . . . 8
6.2 Simulation results . . . 8
6.3 Synthesis . . . 9

7 Conclusion and future work **9**

A Verilog source code **9**
A.1 alu32.v 9
A.2 adder32.v 11
A.3 barrel_shifter32.v 12
A.4 tb_alu32.v 13

1 Introduction

The Arithmetic Logic Unit is the computational core of every processor. It receives two operands and an operation code, and produces a result together with flags that describe the outcome. Every instruction that adds, compares, masks bits or shifts a value is executed by the ALU, so its design decides both the functional behaviour and, to a large extent, the clock speed of a processor.

1.1 Objectives

- Design a 32-bit ALU supporting arithmetic, logic, shift and compare operations.
- Generate the Zero, Negative, Carry and Overflow flags correctly for signed and unsigned data.
- Implement the design in synthesisable Verilog-2005 using a modular, hierarchical structure.
- Verify the design with self-checking testbenches and an independent reference model.

1.2 Tools

Verilog-2005 as the hardware description language; Icarus Verilog and GTKWave for simulation and waveform viewing; Python 3 for the reference model and vector generation; Yosys for synthesis.

2 Theory

2.1 Two's complement arithmetic

Signed integers are stored in two's complement, so the same adder handles signed and unsigned operands. In 32 bits the signed range is -2^{31} to $2^{31}-1$. Subtraction is performed as

$$A - B = A + \overline{B} + 1.$$

2.2 Carry-lookahead addition

In a ripple-carry adder each stage waits for the carry of the previous stage, so the delay grows linearly with the word length. A carry-lookahead adder removes this wait by forming the carries directly from two signals per bit, the *propagate* $p_i = a_i \oplus b_i$ and the *generate* $g_i = a_i b_i$. For a 4-bit block with carry-in c_0:

$$\begin{aligned} c_1 &= g_0 + p_0 c_0 \\ c_2 &= g_1 + p_1 g_0 + p_1 p_0 c_0 \\ c_3 &= g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0 \\ c_4 &= g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0 \\ s_i &= p_i \oplus c_i \end{aligned}$$

The 32-bit adder in this project chains eight such 4-bit blocks. Inside a block the carries are fully looked ahead, while between blocks the carry ripples, which is a common compromise between area and speed.

2.3 Status flags

Flag	Meaning	Condition
Z	Zero	result $= 0$
N	Negative	result[31] $= 1$
C	Carry	carry out of the adder (for subtraction, $C = 1$ means no borrow)
V	Overflow	operands have equal sign, result has the opposite sign

Table 1: Status flags. C and V are defined for ADD, SUB, INC and DEC only.

2.4 Comparison using the subtractor

The compare operations compute $A + \overline{B} + 1$ and inspect the flags. For signed numbers, $A < B$ exactly when $N \oplus V = 1$. For unsigned numbers, $A < B$ exactly when a borrow occurs, that is when $C = 0$. This saves a dedicated comparator.

2.5 Barrel shifter

A barrel shifter shifts by any amount from 0 to 31 positions in a single pass using five cascaded 2:1 multiplexer stages that shift by 1, 2, 4, 8 and 16 positions, each enabled by one bit of the shift amount. Only a right shifter is built. A left shift is obtained by reversing the bit order at the input and again at the output. Arithmetic right shift fills the vacated bits with the sign bit, while logical shifts fill with zero.

3 Design

3.1 Interface

Port	Direction	Width	Description
`a`, `b`	input	32	Operands
`op`	input	4	Operation select
`result`	output	32	ALU result
`zero`	output	1	Result is zero
`negative`	output	1	Result is negative
`carry`	output	1	Carry out (ADD, SUB, INC, DEC)
`overflow`	output	1	Signed overflow (ADD, SUB, INC, DEC)

Table 2: Ports of module `alu32`.

3.2 Operation set

OP	Mnemonic	Function
0000	ADD	$A + B$
0001	SUB	$A - B$
0010	AND	$A \,\&\, B$
0011	OR	$A \mid B$
0100	XOR	$A \oplus B$
0101	NOR	$\sim (A \mid B)$
0110	NAND	$\sim (A \,\&\, B)$
0111	XNOR	$\sim (A \oplus B)$
1000	SLL	$A \ll B[4{:}0]$
1001	SRL	$A \gg B[4{:}0]$, zero fill
1010	SRA	$A \gg B[4{:}0]$, sign fill
1011	SLT	1 if $A < B$ (signed), else 0
1100	SLTU	1 if $A < B$ (unsigned), else 0
1101	INC	$A + 1$
1110	DEC	$A - 1$
1111	NOT	$\sim A$

Table 3: Operation encoding.

3.3 Architecture

Figure 1 shows the top-level structure. The ALU is purely combinational. An operand-conditioning stage feeds the adder with B, $\overline{B}$, 0 or `0xFFFFFFFF` together with the matching carry-in, so that a single adder serves ADD, SUB, INC, DEC, SLT and SLTU. The adder, logic unit, barrel shifter and compare logic work in parallel and a 16:1 multiplexer driven by `op` selects the result. The flag unit takes Z and N from the result and C and V from the adder, with C and V forced to zero unless the operation is an arithmetic one.

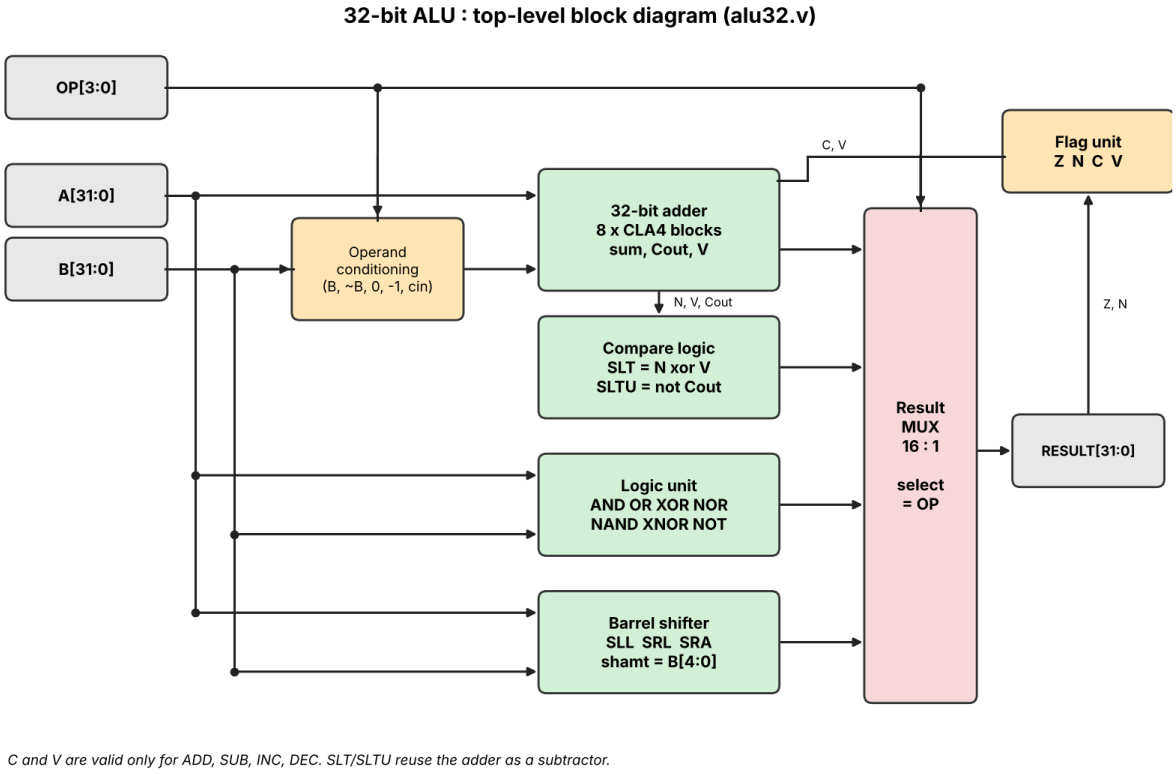


Figure 1: Top-level block diagram of the 32-bit ALU.

Operation	Adder B input	Carry-in
ADD	B	0
SUB, SLT, SLTU	$\overline{B}$	1
INC	0	1
DEC	`0xFFFFFFFF`	0

Table 4: Operand conditioning for the shared adder.

3.4 Adder

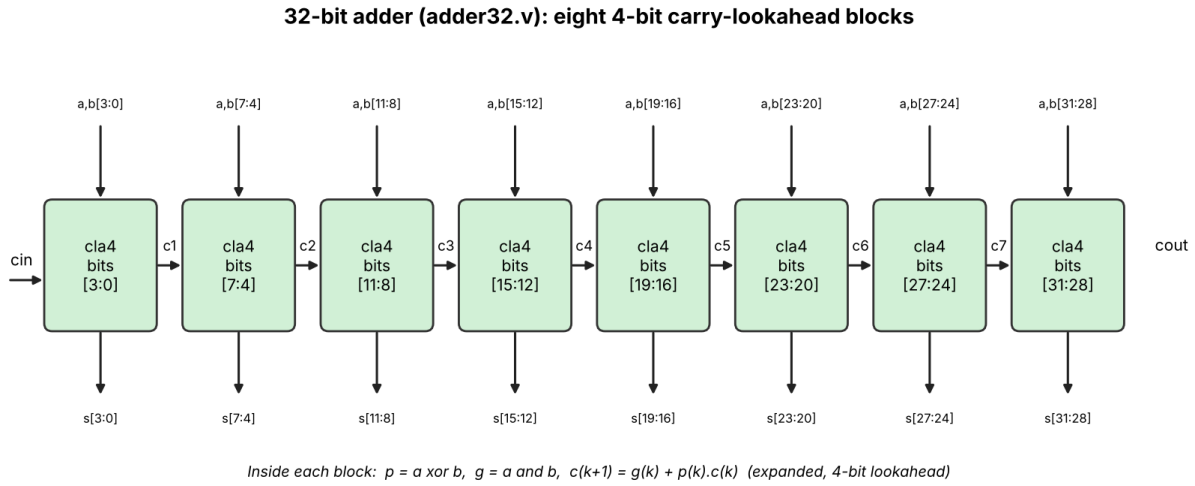


Figure 2: 32-bit adder built from eight 4-bit carry-lookahead blocks.

Signed overflow is detected as $V = (a_{31} = b'_{31}) \wedge (s_{31} \neq a_{31})$, where b' is the operand actually presented to the adder. This one expression is correct for ADD, SUB (where $b' = \overline{B}$), INC and DEC.

3.5 Barrel shifter

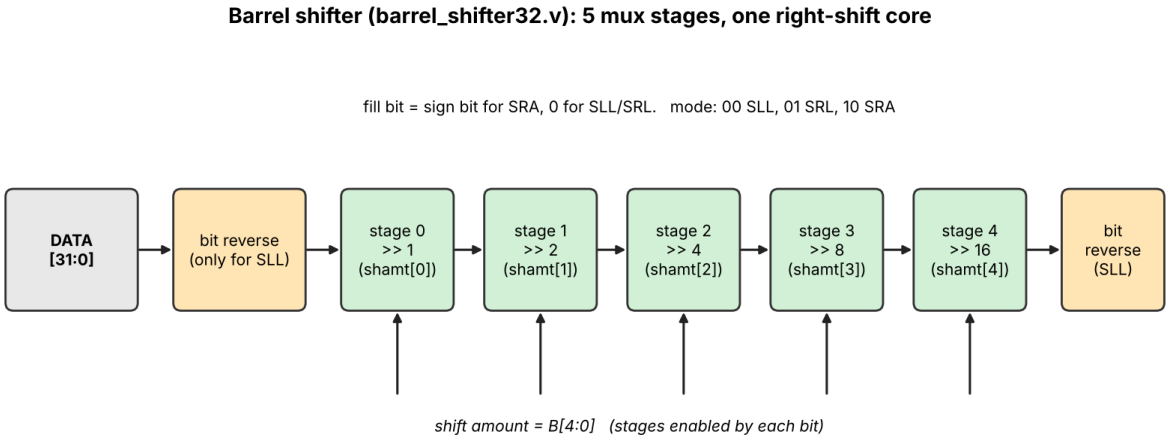


Figure 3: Five-stage barrel shifter with bit reversal for left shifts.

4 Verilog implementation

The design is split into four modules, listed in Table 5. The complete source code is given in Appendix A.

Module	File	Role
`alu32`	`rtl/alu32.v`	Top module: conditioning, result mux, compare, flags
`adder32`	`rtl/adder32.v`	Eight `cla4` blocks chained by carry
`cla4`	`rtl/adder32.v`	4-bit carry-lookahead block
`barrel_shifter32`	`rtl/barrel_shifter32.v`	SLL, SRL and SRA

Table 5: Module hierarchy.

Design decisions worth noting: the adder is structural (a `generate` loop of `cla4` instances) so that the lookahead logic is visible; the result multiplexer is a single `case` statement with a default that returns zero; and the code uses only Verilog-2005 constructs and no vendor primitives, so it is portable across simulators and synthesis tools.

5 Verification

5.1 Method

The system testbench `tb_alu32` contains a behavioural reference model that is deliberately written differently from the RTL. It uses the simulator's own `+`, `-`, shift and comparison operators, and finds signed overflow by comparing bits 33 and 32 of a 34-bit sign-extended result. A second, independent check is provided by a Python model whose output is stored in `tb/vectors.mem` and read by the testbench. For every vector the result and all four flags are compared.

5.2 Test plan

ID	Test	Purpose
T1	Known answers	Sums, differences, carry, overflow, borrow, signed and unsigned compare, SRA
T2	Corner × corner	All 16 operations on every pair of 8 corner values
T3	All shift amounts	SLL, SRL, SRA for shift 0 to 31; upper bits of B ignored
T4	Constrained random	20,000 operations, 25% of operands biased to corner values
T5	Golden vectors	3,100 vectors from the Python model
U1	`cla4` exhaustive	All 512 combinations of 4-bit inputs and carry-in
U2	`adder32` random	Carry ripple across all 8 blocks, 20,000 random additions
U3	`barrel_shifter32`	Every shift amount and mode on several patterns plus random data

Table 6: Test plan.

5.3 Golden-model cross-check

The Python script `sw/ref_model.py` contains two models: a plain-integer reference, and a bit-level emulation of the RTL structure (the `cla4` chain, the five-stage shifter, the $N \oplus V$ comparison and the flag masking). The two agree on all 3,100 golden vectors and on a further 200,000 random vectors, with zero mismatches. This confirms the logic equations used in the Verilog; it does not replace running the Verilog testbenches.

5.4 Known-answer tests

Table 7 lists a representative set of expected results produced by the golden model. These are the values the testbench checks the hardware against.

Op	A	B	Result	N	Z	C	V
ADD	0000000A	00000014	0000001E	0	0	0	0
ADD	FFFFFFFF	00000001	00000000	0	1	1	0
ADD	7FFFFFFF	00000001	80000000	1	0	0	1
SUB	00000032	00000014	0000001E	0	0	1	0
SUB	00000005	0000000A	FFFFFFFB	1	0	0	0
SUB	80000000	00000001	7FFFFFFF	0	0	1	1
AND	F0F0F0F0	0FF00FF0	00F000F0	0	0	0	0
XOR	AAAAAAAA	55555555	FFFFFFFF	1	0	0	0
SLL	00000001	00000004	00000010	0	0	0	0
SRL	80000000	00000004	08000000	0	0	0	0
SRA	80000000	00000004	F8000000	1	0	0	0
SLT	FFFFFFFF	00000001	00000001	0	0	0	0
SLTU	FFFFFFFF	00000001	00000000	0	1	0	0
INC	7FFFFFFF	00000000	80000000	1	0	0	1
DEC	00000000	00000000	FFFFFFFF	1	0	0	0
NOT	0000FFFF	00000000	FFFF0000	1	0	0	0

Table 7: Expected results for selected operations (hexadecimal).

6 Simulation and synthesis

6.1 Running the simulation

From the project root, `sh run_sim.sh` (Linux/macOS) or `run_sim.bat` (Windows) compiles and runs the three testbenches with Icarus Verilog. Each testbench ends with the line `>>> ALL TESTS PASSED <<<` when every check succeeds, and `tb_alu32` also prints a coverage line giving the number of checks per operation. The waveform is saved as `sim/alu32.vcd` and can be opened with GTKWave.

6.2 Simulation results

[Insert screenshot of the testbench console output here]

Figure 4: Console output of `run_sim` showing all tests passed.

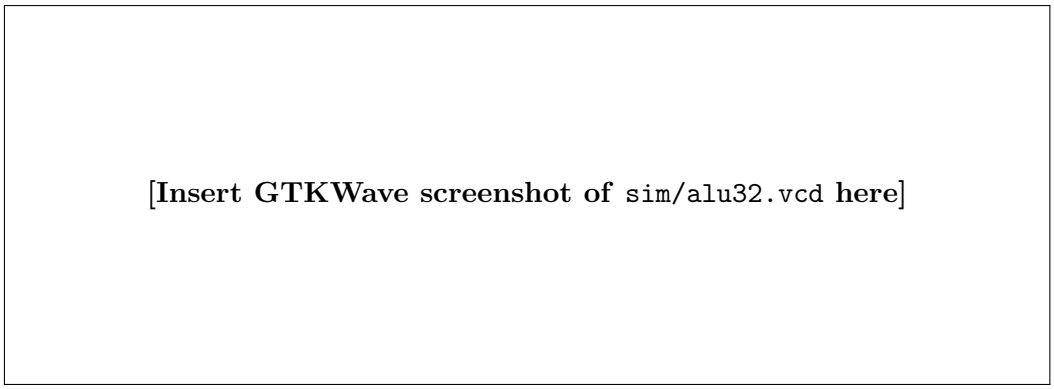


Figure 5: Waveform of `a`, `b`, `op`, `result` and the flags.

6.3 Synthesis

The scripts `synth/synth_generic.ys` and `synth/synth_ecp5.ys` synthesise the design with Yosys to a generic 2-input gate library and to Lattice ECP5 LUTs respectively; `sh run_synth.sh` runs both and writes the statistics to `synth/reports/`.

7 Conclusion and future work

A 32-bit ALU with sixteen operations and four status flags was designed in modular Verilog-2005. Sharing one adder between addition, subtraction, increment, decrement and both comparisons keeps the hardware small, and the carry-lookahead blocks and barrel shifter give predictable, parallel structures. The verification setup combines an independent reference model, directed and random tests, and golden vectors from a second model written in Python.

Possible extensions are a two-level lookahead adder to shorten the carry path, a hardware multiplier and divider, a registered (pipelined) version with a clock, and integration of the ALU into a simple processor datapath.

References

[1] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 5th ed., Pearson, 2013.

[2] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed., Oxford University Press, 2010.

[3] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition*, 2nd ed., Morgan Kaufmann, 2020.

[4] *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005.

[5] C. Wolf, *Yosys Open SYnthesis Suite*, https://yosyshq.net/yosys/.

A Verilog source code

A.1 alu32.v

```
//============================================================
// Authors : Mohammad Irtiqa Bhat (2024-25MECECS005), Mohammad Abbas (2025MECECS002)
// Institute: National Institute of Technology Srinagar, Dept. of Electronics and Communication Engineering
// File    : alu32.v   (TOP MODULE)
// Desc    : Combinational 32-bit ALU, 16 operations, 4 flags.
//   Flag convention (ARM style):
//     C = carry out of the adder (for SUB/DEC, C=1 means NO borrow)
//     V = signed overflow, N = result[31], Z = (result == 0)
//     C and V are only defined for ADD, SUB, INC, DEC (0 otherwise).
//============================================================
`timescale 1ns/1ps

module alu32 (
    input  wire [31:0] a,
    input  wire [31:0] b,
    input  wire [3:0]  op,
    output reg  [31:0] result,
    output wire        zero,
    output wire        negative,
    output wire        carry,
    output wire        overflow
);
    // ---------------- opcode encoding ----------------
    localparam OP_ADD  = 4'b0000;
    localparam OP_SUB  = 4'b0001;
    localparam OP_AND  = 4'b0010;
    localparam OP_OR   = 4'b0011;
    localparam OP_XOR  = 4'b0100;
    localparam OP_NOR  = 4'b0101;
    localparam OP_NAND = 4'b0110;
    localparam OP_XNOR = 4'b0111;
    localparam OP_SLL  = 4'b1000;
    localparam OP_SRL  = 4'b1001;
    localparam OP_SRA  = 4'b1010;
    localparam OP_SLT  = 4'b1011;
    localparam OP_SLTU = 4'b1100;
    localparam OP_INC  = 4'b1101;
    localparam OP_DEC  = 4'b1110;
    localparam OP_NOT  = 4'b1111;

    // ------------- operand conditioning for the adder -------------
    //  ADD : a + b          SUB/SLT/SLTU : a + ~b + 1
    //  INC : a + 0 + 1      DEC          : a + 0xFFFFFFFF + 0
    reg [31:0] add_b;
    reg        add_cin;
    always @(*) begin
        case (op)
            OP_SUB, OP_SLT, OP_SLTU: begin add_b = ~b;           add_cin = 1'b1; end
            OP_INC:                  begin add_b = 32'd0;        add_cin = 1'b1; end
            OP_DEC:                  begin add_b = 32'hFFFFFFFF; add_cin = 1'b0; end
            default:                 begin add_b = b;            add_cin = 1'b0; end
        endcase
    end

    wire [31:0] add_sum;
    wire        add_cout;
    adder32 u_adder (.a(a), .b(add_b), .cin(add_cin), .sum(add_sum), .cout(add_cout));

    // signed overflow: operands have equal sign, result sign differs
    wire add_ovf = (a[31] == add_b[31]) & (add_sum[31] != a[31]);

    // ---------------- barrel shifter ----------------
```

```
    wire [1:0]  sh_mode = (op == OP_SLL) ? 2'b00 :
                          (op == OP_SRL) ? 2'b01 : 2'b10;
    wire [31:0] shift_y;
    barrel_shifter32 u_shift (.data(a), .shamt(b[4:0]), .mode(sh_mode), .y(shift_y));

    // ---------------- compare (re-uses the subtractor) ----------------
    wire slt_bit  = add_sum[31] ^ add_ovf;   // signed   a < b  (N xor V)
    wire sltu_bit = ~add_cout;               // unsigned a < b  (borrow)

    // ---------------- result multiplexer ----------------
    always @(*) begin
        case (op)
            OP_ADD, OP_SUB,
            OP_INC, OP_DEC : result = add_sum;
            OP_AND         : result = a & b;
            OP_OR          : result = a | b;
            OP_XOR         : result = a ^ b;
            OP_NOR         : result = ~(a | b);
            OP_NAND        : result = ~(a & b);
            OP_XNOR        : result = ~(a ^ b);
            OP_SLL, OP_SRL,
            OP_SRA         : result = shift_y;
            OP_SLT         : result = {31'd0, slt_bit};
            OP_SLTU        : result = {31'd0, sltu_bit};
            OP_NOT         : result = ~a;
            default        : result = 32'd0;
        endcase
    end

    // ---------------- status flags ----------------
    wire is_arith = (op == OP_ADD) | (op == OP_SUB) | (op == OP_INC) | (op == OP_DEC);

    assign zero     = (result == 32'd0);
    assign negative = result[31];
    assign carry    = is_arith & add_cout;
    assign overflow = is_arith & add_ovf;
endmodule
```

A.2 adder32.v

```
//=============================================================
// Authors : Mohammad Irtiqa Bhat (2024-25MECECS005), Mohammad Abbas (2025MECECS002)
// Institute: National Institute of Technology Srinagar, Dept. of Electronics and
    Communication Engineering
// File    : adder32.v
// Desc    : 32-bit adder built from eight 4-bit carry-lookahead
//           (CLA) blocks. Carry ripples between blocks.
//=============================================================
`timescale 1ns/1ps

// ---------- 4-bit carry-lookahead block ----------
module cla4 (
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire       cin,
    output wire [3:0] s,
    output wire       cout
);
    wire [3:0] p = a ^ b;   // propagate
    wire [3:0] g = a & b;   // generate

    wire c1 = g[0] | (p[0] & cin);
```

```
    wire c2 = g[1] | (p[1] & g[0]) | (p[1] & p[0] & cin);
    wire c3 = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0])
                   | (p[2] & p[1] & p[0] & cin);
    assign cout = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1])
                       | (p[3] & p[2] & p[1] & g[0])
                       | (p[3] & p[2] & p[1] & p[0] & cin);

    assign s = p ^ {c3, c2, c1, cin};
endmodule

// ---------- 32-bit adder ----------
module adder32 (
    input  wire [31:0] a,
    input  wire [31:0] b,
    input  wire        cin,
    output wire [31:0] sum,
    output wire        cout
);
    wire [8:0] c;
    assign c[0] = cin;

    genvar i;
    generate
        for (i = 0; i < 8; i = i + 1) begin : blk
            cla4 u (.a(a[4*i+3:4*i]), .b(b[4*i+3:4*i]), .cin(c[i]),
                    .s(sum[4*i+3:4*i]), .cout(c[i+1]));
        end
    endgenerate

    assign cout = c[8];
endmodule
```

A.3 barrel_shifter32.v

```
//=============================================================
// Authors : Mohammad Irtiqa Bhat (2024-25MECECS005), Mohammad Abbas (2025MECECS002)
// Institute: National Institute of Technology Srinagar, Dept. of Electronics and
    Communication Engineering
// File    : barrel_shifter32.v
// Desc    : 32-bit barrel shifter, 5 mux stages (shift by 1,2,4,8,16).
//           One right-shifter core is reused for left shifts by
//           reversing the bits at the input and at the output.
// mode    : 00 = SLL, 01 = SRL, 10 = SRA
//=============================================================
`timescale 1ns/1ps

module barrel_shifter32 (
    input  wire [31:0] data,
    input  wire [4:0]  shamt,
    input  wire [1:0]  mode,
    output wire [31:0] y
);
    function [31:0] rev;
        input [31:0] x;
        integer k;
        begin
            for (k = 0; k < 32; k = k + 1)
                rev[k] = x[31-k];
        end
    endfunction

    wire left = (mode == 2'b00);
```

```
    wire fill = (mode == 2'b10) & data[31];   // sign bit only for SRA

    wire [31:0] s0 = left ? rev(data) : data;
    wire [31:0] s1 = shamt[0] ? {     fill,  s0[31:1]}  : s0;
    wire [31:0] s2 = shamt[1] ? {{2{fill}},  s1[31:2]}  : s1;
    wire [31:0] s3 = shamt[2] ? {{4{fill}},  s2[31:4]}  : s2;
    wire [31:0] s4 = shamt[3] ? {{8{fill}},  s3[31:8]}  : s3;
    wire [31:0] s5 = shamt[4] ? {{16{fill}}, s4[31:16]} : s4;

    assign y = left ? rev(s5) : s5;
endmodule
```

A.4 tb_alu32.v

```
//============================================================
// Authors : Mohammad Irtiqa Bhat (2024-25MECECS005), Mohammad Abbas (2025MECECS002)
// Institute: National Institute of Technology Srinagar, Dept. of Electronics and
    Communication Engineering
// File : tb_alu32.v  -- self-checking system testbench for alu32
//   * independent behavioural reference model (task "model")
//   * directed corner tests, all shift amounts, random regression
//   * golden-vector file tb/vectors.mem (made by sw/ref_model.py)
//   * per-operation coverage counters
// Run from the project root:  sh run_sim.sh
//============================================================
`timescale 1ns/1ps

module tb_alu32;
    parameter NV = 3100;

    reg  [31:0] a, b;
    reg  [3:0]  op;
    wire [31:0] result;
    wire        zero, negative, carry, overflow;

    integer errors = 0, checks = 0, i, j, seed = 32'h1234ABCD;
    integer cov [0:15];
    reg [31:0] corner [0:7];
    reg [103:0] vec_mem [0:NV-1];
    reg [31:0] e_res;
    reg        e_c, e_v;

    alu32 dut (.a(a), .b(b), .op(op), .result(result), .zero(zero),
               .negative(negative), .carry(carry), .overflow(overflow));

    // ---------- reference model (different method from the RTL) ----------
    task model;
        input  [3:0]  o;
        input  [31:0] x, y;
        output [31:0] r;
        output        c, v;
        reg [32:0] s33;
        reg [33:0] s34;
        reg signed [31:0] sx, sy;
        begin
            c = 0; v = 0; r = 0; sx = x; sy = y;
            case (o)
                4'b0000: begin
                    s33 = {1'b0, x} + {1'b0, y};  r = s33[31:0];  c = s33[32];
                    s34 = {x[31], x[31], x} + {y[31], y[31], y};  v = s34[33] ^ s34[32];
                end
                4'b0001: begin
```

```
48                  r = x - y;  c = (x >= y);
49                  s34 = {x[31], x[31], x} - {y[31], y[31], y};  v = s34[33] ^ s34[32];
50              end
51              4'b0010: r = x & y;
52              4'b0011: r = x | y;
53              4'b0100: r = x ^ y;
54              4'b0101: r = ~(x | y);
55              4'b0110: r = ~(x & y);
56              4'b0111: r = ~(x ^ y);
57              4'b1000: r = x << y[4:0];
58              4'b1001: r = x >> y[4:0];
59              4'b1010: r = sx >>> y[4:0];
60              4'b1011: r = (sx < sy) ? 32'd1 : 32'd0;
61              4'b1100: r = (x < y)   ? 32'd1 : 32'd0;
62              4'b1101: begin r = x + 32'd1; c = (x == 32'hFFFFFFFF); v = (x == 32'
                    h7FFFFFFF); end
63              4'b1110: begin r = x - 32'd1; c = (x != 32'd0);        v = (x == 32'
                    h80000000); end
64              4'b1111: r = ~x;
65          endcase
66      end
67  endtask
68
69  // ---------- apply one vector and compare ----------
70  task check;
71      input [3:0]  o;
72      input [31:0] x, y;
73      begin
74          op = o; a = x; b = y;
75          #5;
76          model(o, x, y, e_res, e_c, e_v);
77          checks = checks + 1;
78          cov[o] = cov[o] + 1;
79          if (result !== e_res || carry !== e_c || overflow !== e_v ||
80              zero !== (e_res == 32'd0) || negative !== e_res[31]) begin
81              errors = errors + 1;
82              if (errors <= 20)
83                  $display("FAIL op=%b a=%h b=%h : res=%h (exp %h) C=%b/%b V=%b/%b",
84                           o, x, y, result, e_res, carry, e_c, overflow, e_v);
85          end
86      end
87  endtask
88
89  function [31:0] pick;
90      input dummy;
91      begin
92          if (($random(seed) & 3) == 0) pick = corner[$random(seed) & 7];
93          else                          pick = $random(seed);
94      end
95  endfunction
96
97  initial begin
98      $dumpfile("sim/alu32.vcd");
99      $dumpvars(0, tb_alu32);
100
101     for (i = 0; i < 16; i = i + 1) cov[i] = 0;
102     corner[0] = 32'h00000000; corner[1] = 32'h00000001;
103     corner[2] = 32'hFFFFFFFF; corner[3] = 32'h7FFFFFFF;
104     corner[4] = 32'h80000000; corner[5] = 32'hAAAAAAAA;
105     corner[6] = 32'h55555555; corner[7] = 32'h0000FFFF;
106
107     // T1 : known-answer tests
108     check(4'b0000, 32'd10, 32'd20);
109     check(4'b0001, 32'd50, 32'd20);
```

```
110         check(4'b0000, 32'hFFFFFFFF, 32'd1);          // carry out
111         check(4'b0000, 32'h7FFFFFFF, 32'd1);          // signed overflow
112         check(4'b0001, 32'h80000000, 32'd1);          // signed overflow
113         check(4'b0001, 32'd5, 32'd10);                // borrow
114         check(4'b1011, 32'hFFFFFFFF, 32'd1);          // -1 < 1 (signed)
115         check(4'b1100, 32'hFFFFFFFF, 32'd1);          // not < (unsigned)
116         check(4'b1010, 32'h80000000, 32'd4);          // SRA keeps sign
117         $display("T1 known-answer          done  (errors so far: %0d)", errors);
118
119         // T2 : every operation on every corner pair
120         for (i = 0; i < 16; i = i + 1)
121             for (j = 0; j < 64; j = j + 1)
122                 check(i[3:0], corner[j / 8], corner[j % 8]);
123         $display("T2 corner x corner       done  (errors so far: %0d)", errors);
124
125         // T3 : all 32 shift amounts for SLL/SRL/SRA
126         for (i = 8; i < 11; i = i + 1)
127             for (j = 0; j < 32; j = j + 1) begin
128                 check(i[3:0], 32'h80000001, j);
129                 check(i[3:0], 32'h12345678, j);
130                 check(i[3:0], 32'hF0F0F0F0, {27'd0, j[4:0]} | 32'hFFFFFFE0); // upper
                        bits ignored
131             end
132         $display("T3 all shift amounts     done  (errors so far: %0d)", errors);
133
134         // T4 : constrained random regression
135         for (i = 0; i < 20000; i = i + 1)
136             check($random(seed), pick(0), pick(0));
137         $display("T4 random (20000)        done  (errors so far: %0d)", errors);
138
139         // T5 : golden vectors from the Python model
140         for (i = 0; i < NV; i = i + 1) vec_mem[i] = {104{1'bx}};
141         $readmemh("tb/vectors.mem", vec_mem);
142         for (i = 0; i < NV; i = i + 1)
143             if ((^vec_mem[i]) !== 1'bx) begin
144                 op = vec_mem[i][103:100]; a = vec_mem[i][99:68]; b = vec_mem[i][67:36];
145                 #5;
146                 checks = checks + 1;
147                 if (result !== vec_mem[i][35:4] ||
148                     {negative, zero, carry, overflow} !== vec_mem[i][3:0]) begin
149                     errors = errors + 1;
150                     if (errors <= 20)
151                         $display("FAIL(vec %0d) op=%b a=%h b=%h : res=%h exp=%h flags=%b
                                exp=%b",
152                                  i, op, a, b, result, vec_mem[i][35:4],
153                                  {negative, zero, carry, overflow}, vec_mem[i][3:0]);
154                 end
155             end
156         $display("T5 golden vectors        done  (errors so far: %0d)", errors);
157
158         $display("------------------------------------------------");
159         $display("FUNCTIONAL COVERAGE (checks per opcode)");
160         $display("ADD=%0d SUB=%0d AND=%0d OR=%0d XOR=%0d NOR=%0d NAND=%0d XNOR=%0d",
161                  cov[0], cov[1], cov[2], cov[3], cov[4], cov[5], cov[6], cov[7]);
162         $display("SLL=%0d SRL=%0d SRA=%0d SLT=%0d SLTU=%0d INC=%0d DEC=%0d NOT=%0d",
163                  cov[8], cov[9], cov[10], cov[11], cov[12], cov[13], cov[14], cov[15]);
164         $display("checks performed : %0d", checks);
165         $display("errors           : %0d", errors);
166         if (errors == 0) $display(">>> ALL TESTS PASSED <<<");
167         else             $display(">>> TESTS FAILED <<<");
168         $display("------------------------------------------------");
169         $finish;
170     end
171 endmodule
```